Native marked-scalar correctness replay

T-1888. Planning PR: https://github.com/isogeny-labs/crypto/pull/3674. Native qualification after initial PR publication; no comparative timing. The original goal remains an end-to-end improvement over regular scalar multiplication.

Exact fixture coefficients and marked scalar labels
Native parser preserves integers beyond binary64 precision

PASS: full map identities and 50 window/input cases
Forward + dual + closing isomorphism + scalar correction

Next: frozen matched native end-to-end comparison
A/A, isolation, held-out points and complete setup accounting

The native C++ implementation reads exact P-384 and P-521 curve/map coefficients and order-seven labels. It evaluates the forward rational map, normalized opposite map, closing isomorphism, and centered scalar correction. OpenSSL provides BIGNUM field arithmetic and curve operations. No Python process participates in execution or replay.

For each target, both full polynomial curve identities pass. Source points are O, G, 2G, 17G and -G. Signed-window widths 3,4,5,6,7 are each checked against OpenSSL's scalar result. All 50 curve/input/window cases pass for both the direct and relation routes. The closing pair agrees with multiplication by its degree. The primitive seventh-root and correcting scalar congruence pass. The OpenSSL public multiscalar configuration agrees too. An altered scalar label is rejected on every nonzero control point.

The fixture reader preserves large integer tokens lexically rather than converting them to floating point. Its supported fixture schema is integer JSON. It rejects duplicate object keys, empty tokens, unsupported numeric types and non-ASCII escaped code points; it is not proposed as a general-purpose JSON library.

The first build succeeded with indentation warnings. The second version fixes those warnings and adds explicit rejection of empty JSON tokens. The second build is warning-free. Both build logs are retained. Compiler: GCC11.5.0; linked backend: OpenSSL3.5.8. Commands: make -j1 under a 600-second cap, then native –qualify fixtures under a 120-second cap. Address-space cap: 2GiB; one process; no cloud submission.

This is qualification on the supplied public known-answer cases, plus exact polynomial identities. There is no timing result, native speedup, general scalar-multiplication advantage or completed end-to-end claim. The binary deliberately has no benchmark mode yet. Baseline source and binary digests are retained before speed optimization. Setup, maps, closure, correction, conversion and verification remain in the benchmark accounting boundary.

Evidence: receipt.json, replay.jsonl, replay.stderr, build.log, source-and-binary.sha256 and the preserved source directory. Historical Python certificates remain unchanged inputs; they are not native runtime results. The report is generated by the native Typst0.13.1 renderer. The next stage requires committed native timing/admission source, a frozen execution protocol, paired A/A and candidate/reference runs, and an independently generated held-out point panel.